

Intégration SMS dans Home Assistant

Vue d'ensemble

La passerelle LilyGO publie et souscrit des messages MQTT selon le schéma suivant :

```

sms/incoming → automation
Téléphone / SIM → MQTT
→ Home → ← Broker ←
Assistant → sms/send → script

```

Topic MQTT	Direction	Contenu
sms/incoming	gateway → HA	{"sender":"+32499...", "message":"texte",
sms/send	HA → gateway	{"recipient":"+32499...", "message":"text
sms/status	gateway → HA	{"state":"online", "rssi":-72, "uptime_s

1. Pré-requis

Intégration MQTT configurée dans HA (Paramètres → Intégrations → MQTT)
Pointer sur le même broker que la passerelle.

Fichiers du projet copiés dans votre répertoire de config HA :

```

config/ packages/ sms_gateway.yaml automations/ 
sms_alerts.yaml sms_commands.yaml

```

2. Installation du package

2.1 Activer les packages dans configuration.yaml

```
homeassistant: packages: sms_gateway: !include packages/sms_gateway.yaml
```

2.2 Inclure les automatisations dans automations.yaml

```
- !include automations/sms_alerts.yaml - !include automations/sms_commands.yaml
```

2.3 Recharger la configuration

Outils développeur → YAML → Tout recharger

ou redémarrer Home Assistant.

3. Capteurs disponibles après installation

Entité	Description
<code>sensor.sms_gateway_etat</code>	online / offline
<code>sensor.sms_gateway_rssi</code>	Signal SIM en dBm
<code>sensor.sms_gateway_uptime</code>	Uptime en secondes
<code>sensor.dernier_sms_expéditeur</code>	Numéro de l'expéditeur
<code>sensor.dernier_sms_message</code>	Contenu du dernier SMS reçu

4. Envoyer un SMS depuis une automation

4.1 Via le script `send_sms` (méthode recommandée)

```
action: - action: script.send_sms data: recipient: "+32499000000" message:
  "Alerte : pH trop bas – {{ states('sensor.aquarium_ph') }}"
```

4.2 Via `mqtt.publish` directement

```
action: - action: mqtt.publish data: topic: sms/send payload: >- {"recipient":
  "+32499000000", "message": "Alerte température : {{
  states('sensor.aquarium_temperature') }} °C"}
```

5. Exemples d'automations d'alerte

5.1 Alerte pH bas (aquarium)

```
- alias: "SMS – pH bas aquarium" trigger: - platform: numeric_state entity_id:
sensor.aquarium_ph below: 6.5 for: minutes: 5 condition: - condition: state
entity_id: sensor.sms_gateway_etat state: "online" action: - action:
script.send_sms data: recipient: "+32499000000" message: >- ■■ pH BAS : {{
states('sensor.aquarium_ph')} } – {{ now().strftime('%H:%M')} } mode: single
```

mode: single empêche l'envoi répété si le pH reste bas.

Utiliser mode: single pour les alertes continues, mode: restart si on veut remettre à jour le message.

5.2 Alerte température hors plage

```
- alias: "SMS – Température aquarium" trigger: - platform: numeric_state
entity_id: sensor.aquarium_temperature below: 24 - platform: numeric_state
entity_id: sensor.aquarium_temperature above: 28 action: - action:
script.send_sms data: recipient: "+32499000000" message: "Température : {{
states('sensor.aquarium_temperature')} } °C (plage 24-28)" mode: single
```

5.3 Alerte panne (gateway offline)

```
- alias: "SMS Gateway hors ligne" trigger: - platform: state entity_id:
sensor.sms_gateway_etat to: "offline" for: minutes: 2 action: - action:
notify.persistent_notification data: title: "SMS Gateway offline" message: "La
passerelle LilyGO ne répond plus."
```

6. Recevoir des commandes SMS

L'automation `sms_commands.yaml` écoute `sms/incoming` et réagit aux mots-clés.

Commandes disponibles (insensibles à la casse)

SMS envoyé	Action dans HA	Réponse SMS
STATUS	Lecture des capteurs	pH + Temp + état pompe
POMPE ON	<code>switch.turn_on</code>	Confirmation
POMPE OFF	<code>switch.turn_off</code>	Confirmation

LUMIERES ON	light.turn_on	Confirmation
LUMIERES OFF	light.turn_off	Confirmation
Autre	—	Liste des commandes

Ajouter une nouvelle commande

Dans `sms_commands.yaml`, ajouter un bloc dans la section `choose` :

```
- conditions: - condition: template value_template: "{{ cmd == 'MA_COMMANDE' }}"
sequence: - action: mon_domaine.mon_action target: entity_id: switch.mon_device
- action: script.send_sms data: recipient: "{{ sender }}" message: "■ Action effectuée."
```

7. Sécurité et bonnes pratiques

Pratique	Raison
Filtrer les expéditeurs autorisés	N'importe qui ayant le numéro de la SIM peut envoyer des commandes
Vérifier que la gateway est <code>online</code> avant d'envoyer	Évite les alertes silencieuses
Limiter la longueur des messages à 160 caractères	Au-delà, le SMS est fragmenté en plusieurs parties
Utiliser <code>mode: single</code> sur les alertes continues	Évite le spam SMS si la condition persiste

Filtrer les expéditeurs (exemple)

```
- alias: "SMS – Dispatcher commandes" trigger: - platform: mqtt topic: sms/incoming condition: - condition: template value_template: >- {{ trigger.payload_json.sender in ['+32499000000', '+32470111222'] }} # ... reste de l'automation
```

8. Dépannage

Symptôme	Vérification
<code>script.send_sms</code> ne fait rien	Vérifier que <code>sensor.sms_gateway_etat</code> est online
SMS envoyé mais non reçu	Vérifier les logs MQTT broker (topic <code>sms/send</code>)
Aucun capteur disponible	Vérifier l'inclusion du package et recharger YAML
Commandes SMS ignorées	Vérifier que l'automation <code>sms_commands</code> est activée
Automation déclenchée plusieurs fois	Ajouter <code>mode: single</code> et un délai <code>for:</code> sur le trigger